arXiv:2102.03313v1 [cs.LG] 5 Feb 2021

On The Connection of Benford's Law and Neural Networks

Surya Kant Sahu, **Abhinav Java** and **Arshad Shaikh**
{surya.oju, java.abhinav99, arshaikh5775}@gmail.com

Abstract

Benford's law, also called Significant Digit Law, is observed in many naturally occurring data-sets. For instance, the physical constants such as Gravitational, Coulomb's Constant, etc., follow this law. In this paper, we define a score, MLH, for how closely a Neural Network's Weights match Benford's law. We show that Neural Network Weights follow Benford's Law regardless of the initialization method. We make a striking connection between Generalization and the MLH of the network. We provide evidence that several architectures from AlexNet to ResNeXt trained on ImageNet, Transformers (BERT, Electra, etc.), and other pre-trained models on a wide variety of tasks have a strong correlation between their test performance and the MLH. We also investigate the influence of Data in the Weights to explain why NNs possibly follow Benford's Law. With repeated experiments on multiple datasets using MLPs, CNNs, and LSTMs. We provide empirical evidence that there is a connection between MLH while training, overfitting. Understanding this connection between Benford's Law and Neural Networks promises a better comprehension of the latter.

1 Introduction and Related Works

The leftmost non-zero digit's occurrence in the observations of a population is not uniformly distributed for many datasets. Instead, it is log-uniform, with 1 occurring the maximum number of times, followed by 2, 3, ... 9. According to Benford's Law [Benford, 1938], the probability for a sample having a significant digit d is given as follows:

$$P(d) = log_{10}(d + \frac{1}{d}), d = 1, 2, 3, ..., 9 \quad (1)$$

Benford's Law (BL) has been observed in many naturally occurring populations, including the physical constants, populations of countries, areas of lakes, stock market indices, tax accounts, etc. [Shao and Ma, 2010]. Attempts have been made to explain the underlying reason for Benford's Law's emergence for specific domains. However, a universally accepted explanation does not yet exist.

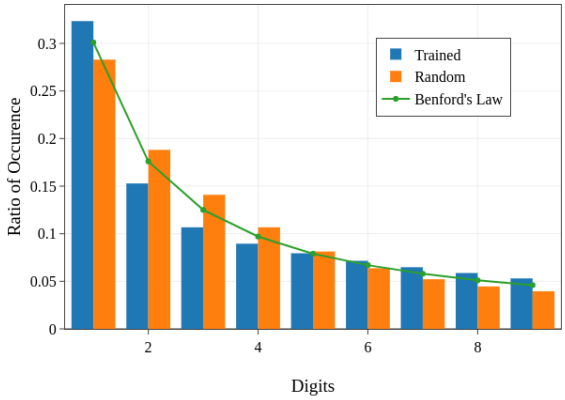


Figure 1: Mantissa Distribution for ResNet152, randomly initialized and trained on ImageNet

Benford's Law has been used to ascertain fraud in taxing and accounting and machine learning literature for Anomaly Detection [Bonettini *et al.*, 2020]. Researchers have also discovered the presence of this law in natural sciences [Sambridge *et al.*, 2010], image gradient magnitude [Jolion, 2001], synthetic and natural images [Acebo and Sbert, 2005], etc.

Incorporating the inductive bias that class labels follow BL in supervised classification tasks in some datasets lead to better-performing classifiers.

The widespread use of this law on datasets and naturally occurring distributions motivates the investigation of the behavior of significant digits of the weights of a neural network. To that end, we show that the weights of neural networks indeed follow this law. We also propose a metric, MLH, based on Benford's Law as a measure of the "goodness" of a network and show empirical observations on how various network performance metrics are closely related to it.

By means of this paper, we sought the answer to the question "Do Neural Network Weights follow Benford's Law?". To do the same, we first describe a simple method to count the frequencies of Significant Digits, hence the Mantissa Dis-

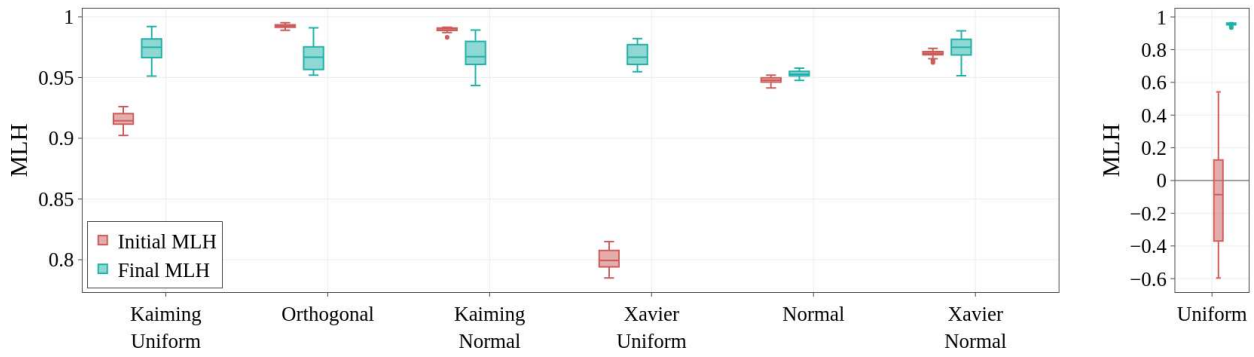
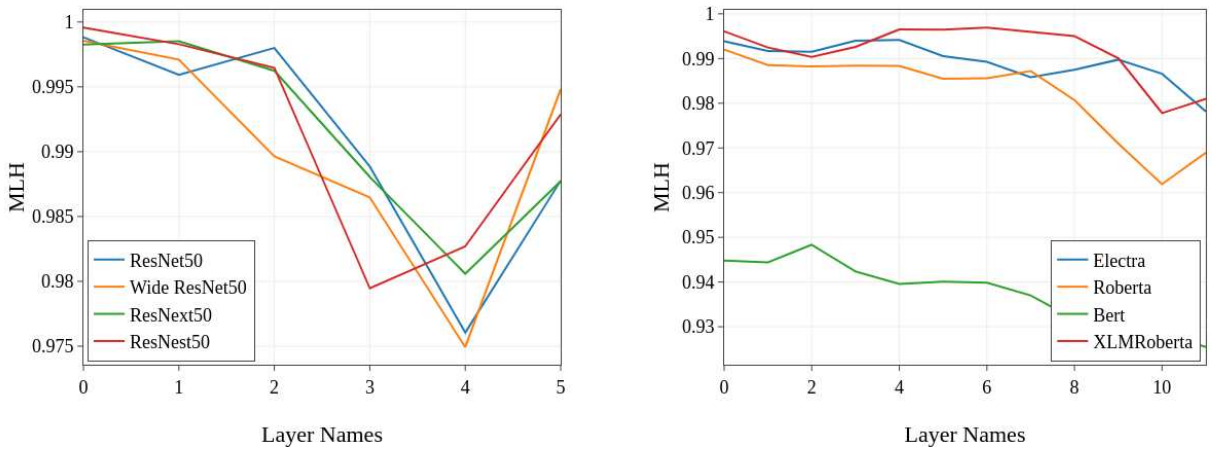


Figure 2: MLH Scores for Trained and Randomly-Initialized Models



(a) Image Classification

(b) Transformers

Figure 3: MLH Score across different layers of models.

tribution for any tensor with floating-point elements.

This is followed by measuring the "closeness" of several representative architectures from both natural language processing and computer vision domain to Benford's Law. This lead to some surprising findings, such as this measure oscillates around a particular point while training; making one to believe that Fermi-Dirac & Boltzmann-Gibbs Distributions, which is known to be followed by many macro and microsystems and known to fluctuate around Benford's Law [Shao and Ma, 2010], and Neural Network Weights and Optimization analogous to the latter.

We investigate whether the correlation with Benford's Law is the same for all architectures and whether this match describes anything about the architecture itself. We find that there is strong evidence against the Null Hypothesis. This work introduces a new perspective to understanding model weight statistics, and it's performance.

2 Neural Nets Follow Benford's Law

We devise a new metric, MLH, that provides the measure of correlation between Benford's Law and the distribution of significant digits of model weights.

This score indicates a strong correlation between the weights and BL and proves to be an essential metric for a wide range of neural networks, and is described in the following sections.

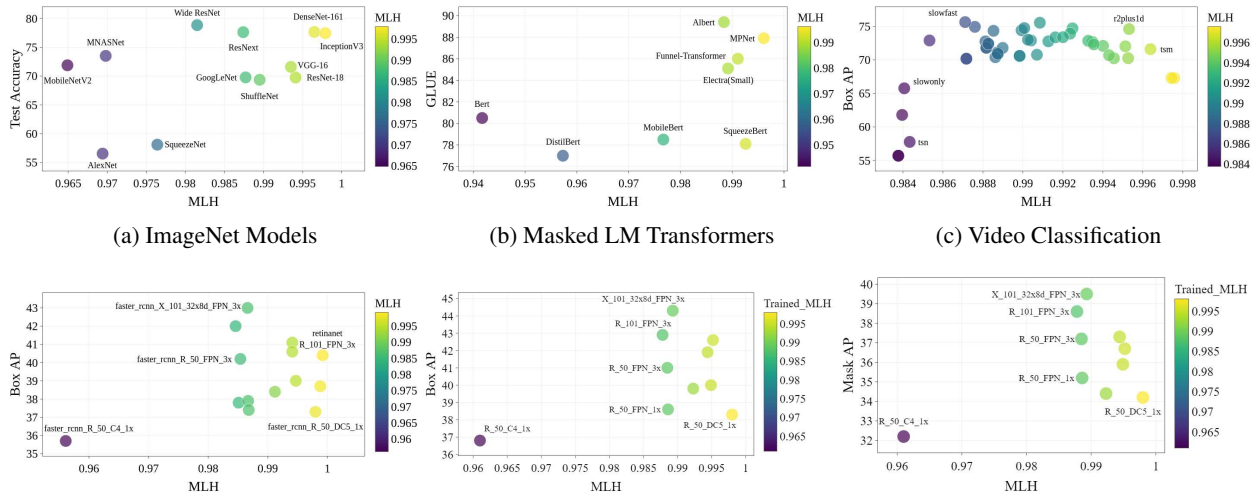
The proposed score, MLH based on the Pearson's Correlation Coefficient [Pearson, 1895] is defined as follows:

$$MLH(\theta) = PearsonR(BinCount(\theta), \beta) \tag{2}$$

$BinCount(\theta)$ is the distribution of Significant Digits of network parameter set θ. β is the distribution defined by BL.

$$BinCount(\theta) = \frac{[f_0, f_1, ..., f_9]}{D_\theta} \tag{3}$$

Here, f_k is is the frequency of significant digit k occurring in θ, D_θ is the dimensionality of θ. We did not include parameters that are initialized with a constant value, such as Bias and BatchNorm parameters.



(a) ImageNet Models (b) Masked LM Transformers (c) Video Classification

(d) Object Detection Models (e) Instance Segmentation vs Box AP (f) Instance Segmentation vs Mask AP

Figure 4: MLH Score of several representative architectures against respective Performance Metrics. For full list of models and computed values please refer the Appendix.

In our implementation, we multiply the weights by a constant 10^{10} then take the first non-zero digit. Note that multiplying with a constant scalar still holds BL due to its property of *Scale Invariance* [Hill, 1995b]. We provide PyTorch implementation in the Appendix and supplementary material along with code explanation.

The formulation of MLH metric gives an easy way to compute the compute the p-values for the match of the distributions.

In Figure1, We compare the Mantissa Distribution of a ResNet152 trained on ImageNet and an uninitialized ResNet152 (Glorot Normal; Bias and BatchNorm are initialized with a constant). Using $BinCount(.)$, we can compute the Mantissa Distribution for the Weights. We see that both are very close to Benford's Law (>0.99 Pearson's R).

In Figure2, we compute MLH for various pre-trained convolutional neural networks and Transformers and plot their layer-wise MLH. We see that generally, the MLH is high at the first layers, decreases gradually, but spikes again at the last layer.

To show that a Neural Net's match with BL is high not just because of the virtue of the initialization method. In Fig. 2, we show results of an experiment where we repeatedly train 20 networks each for each initialization method, including initializing sub-optimally; we show that the MLH after training is always high regardless of the initial MLH.

In this set of experiments, the default values of PyTorch were used for Initialization methods.

3 MLH and Generalization

In this section, we present the relationship between the generalization of a given neural network with it's MLH score.

Firstly, we compute MLH for various open-source models available in PyTorch [Falcon, 2019], MMAction2 [Contributors, 2020], Detectron2 [Wu *et al.*, 2019], HuggingFace's Transformers [Wolf *et al.*, 2020] Libraries [Wolf *et al.*, 2020], and compute Pearson's Correlation Coefficients for a common reported metric. The Table 1 and Fig 4 have the results.

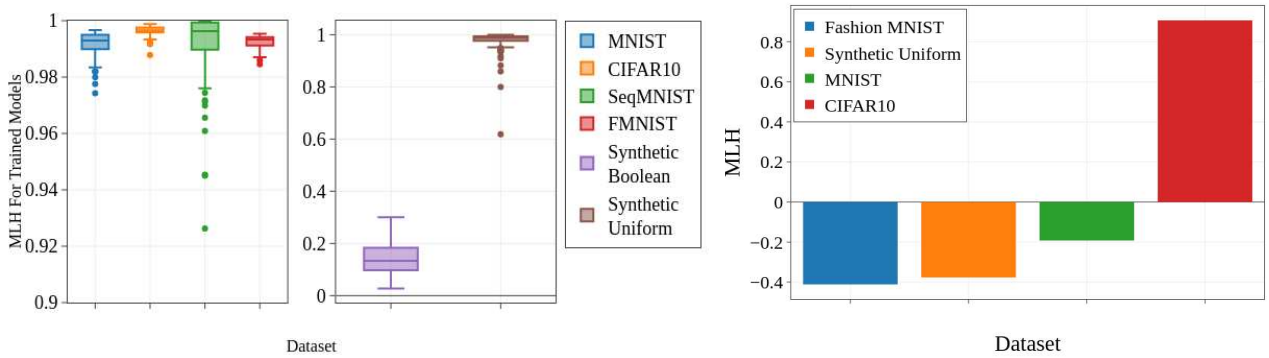
The pre-trained models are chosen from three significant domains, image, video, and text. Test accuracy on ImageNet and Kinetics-400 is selected as a metric of comparison, and GLUE score [Wang *et al.*, 2018] is considered for the Masked LM networks and perplexity(PPL) for the Causal LM networks. MLH for a model is computed by calculating the *mantissa distribution* for the whole network by aggregating the count vector across all weight tensors.

We observe that models like AlexNet [Krizhevsky *et al.*, 2017] and SqueezeNet [Iandola *et al.*, 2016], which have lower test accuracy, show lower MLH scores than models like DenseNet-161 and Inception-v3 that have higher test accuracy higher MLH score.

Similarly, we do the same for Masked LMs, Causal LMs, Object Detectors, Instance Segmentation Models; we compute correlation w.r.t reported GLUE scores. Again, we see that the MLH is positively correlated with a value of 0.5525 for Masked LMs.

A positive slope is observed in both of the plots in Figure4. By considering the positive slopes and the positive correlations, it is anticipated that models that perform well, i.e., models that have a better test result, tend to have a better MLH.

To understand how the MLH score varies over different layers of models, we also plot the layer-wise correlation and observe a trend for a class of models. As illustrated in Figure 3, the general trend across all neural networks indicates that the MLH score of the initial layers is the highest and decreases upon progressing deeper. A sudden spike in the MLH score is noticed in the image classification models at the fully connected layer, unlike the transformer models. Note that this property of models trained on ImageNet is con-



(a) MLH of Models

(b) MLH of Datasets

Figure 5: MLH for Datasets and Networks Trained on them.

Task	Metric	R	*p*-value	Support
Image Classif.	ACC	0.4526	.14	12
Video Classif.	ACC	0.4312	.03	46
Masked LM	GLUE	0.5525	.15	8
Causal LM	PPL	0.3857	.45	6
Object Detection	Box AP	0.3740	.19	14
Instance Seg.	Box AP	0.4441	.20	10
Instance Seg.	Mask AP	0.4526	.19	10

Table 1: Pearson's R between performance metrics of several pre-trained models and their MLH along with the *p*-values. The detailed list of models can be found in Figure 4 and the Appendix.

sistent across all representative models, including VGG19 [Simonyan and Zisserman, 2014], InceptionV3 [Szegedy *et al.*, 2016], etc. (The plots of the remaining models have been left out for the sake of brevity and ease of representation).

Causal LMs have a positive correlation with Perplexity (lower is better), which is an odd-ball. A point to note is that this correlation's support is just six samples due to the lack of open-source Causal Language Models in PyTorch. Also, words and natural language text are known to be closely related to Zipf's Law rather than Benford's Law [Corral *et al.*, 2015], and computing Mantissa Distribution for sequences and words is not trivial. Still, the same reasoning would follow for Masked LMs as well. This observation will be a part of our future work.

We conclude that,

MLH could be an Indicator of Generalization.

4 Effect of Data on MLH

It is well known that RGB Images and their pixel values follow Benford's Law [Bonettini *et al.*, 2020]. We investigate whether the data distribution affects the Network Weights, causing them to match Benford's Law.

We investigate CIFAR10 [Krizhevsky and others, 2009], MNIST [LeCun and Cortes, 2010], FashionMNIST [Xiao *et al.*, 2017], Sequential MNIST (SeqMNIST) [Goodfellow *et al.*, 2013], two Synthetically Generated Datasets, and 100 Nodels trained[1] on each of these datasets. We train a small LeNet-inspired network for MNIST and FashionMNIST, DenseNet121 for CIFAR10, LSTM [Hochreiter and Schmidhuber, 1997a] for SeqMNIST, and MLP for Synthetic Datasets.

For Synthetic-Boolean Dataset, We generate a random boolean function taking the $argmax$ over a randomly initialized Network with 64 boolean input features. Since the inputs are boolean, i.e., Base 2, Benford's Law for Base 2 is 1 [Hill, 1995a].

For Synthetic-Uniform Dataset, we do the same as above, but the inputs are drawn uniformly from the range $[0, 1]$.

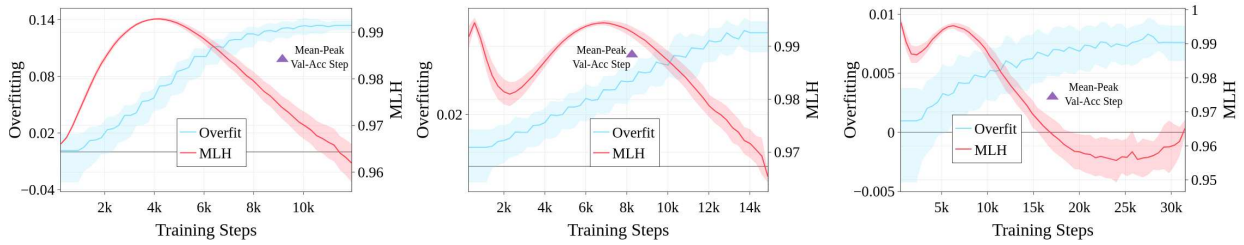
Since the Scale Invariance Property of BL holds, we only divide by 255 for Image Datasets and do not normalize this experiment's features.

As Observed in Figure5, CIFAR10 has positive MLH, and others have Negative MLH, whereas all of the models except for the ones trained on Synthetic-Boolean have >0.9 median MLH. Models trained on Synthetic-Boolean have a low but positive median MLH of about 0.2. We believe that MLH of the network is more related to feature variance than MLH of the Data.

With this, we conclude that,

High Dataset MLH does not necessarily mean High Network MLH.

[1]Experiments were conducted in an Ubuntu system, PyTorch 1.7, with a single NVIDIA RTX 2060 GPU, 16GB of RAM



(a) CIFAR10 (b) FMNIST (c) MNIST

Figure 6: MLH Score and degree of overfitting for 100 models, and how it changes over training steps for multiple datasets.

5 MLH and Overfitting

Neural networks are a potent tool capable of understanding the relationship and identifying patterns from the given data. This has primarily been possible due to the representative capacity of deep neural networks. However, as model complexity increases, the networks tend to overfit. Overfitting remains an open issue in the deep learning community since it's mitigation is still largely heuristic.

The validation accuracy curve during training time is a reasonable indicator of high variance. When a model fits the training data increasingly well, its accuracy on the validation dataset starts dropping, and we can infer that the model is attempting to "remember" the data.

We define $Overfit$ at time t as the difference of Train and Validation accuracy.

$$Overfit_t = Accuracy_t^{train} - Accuracy_t^{val} \tag{4}$$

We plot the relationship between overfitting and MLH, and infer that MLH score is a good indicator of overfitting.

As illustrated in Figure 6, a clear drop in the MLH score is observed as the overfit curve's slope reaches its peak value. Also, note that the MLH score tends to oscillate towards the beginning, and then it reaches a crest that stipulates the saturation point. After this point, the MLH curve wavelength increases rapidly with a decrease in MLH score and an increase in the degree of overfitting.

As soon as the model starts overfitting, the slope of the curve representing MLH score starts decreasing. This relationship is clearly illustrated in Figure 6. This is a vital observation since it eliminates the need for a validation dataset, potentially allowing more training data.

For this experiment's purposes, we consider three image classification datasets, i.e., MNIST, FashionMNIST (FMNIST), and CIFAR10, and train 100 models for each of the three datasets. High early stopping patience of 25 has been set for this experiment to ensure that the trained models overfit to a certain degree.

LeNet-like [LeCun *et al.*, 1998] architecture with two blocks of Strided Convolution-LeakyReLU followed by a classification head is trained for the experiment on MNIST and FMNIST datasets. We use the DenseNet121 [Huang *et al.*, 2017] for a sufficiently large model is required to overfit on CIFAR10 dataset.

The experimental setting and empirical results agree prominently. This can be noted by comparing Figure 6(a) with Figure 6(b) and Figure 6(c). Note that the model trained on CIFAR10 is much larger in terms of parameters (DenseNet) than the one trained on MNIST and Fashion MNIST (LeNet), making it far more susceptible to overfitting, which can also be seen clearly.

Hence, we conclude that

MLH is related to Optimization and Overfitting.

In our future work, we plan on using MLH as an indication for Early Stopping eliminating the need for a Validation dataset. There has been prior work where metric based on local statistics of gradients have been used for similar purposes [Mahsereci *et al.*, 2017].

6 Conclusion

We start with the simple question of whether Neural Network Weights obey Benford's Law, and we find that is, indeed, the case. Furthermore, we define a metric MLH that encapsulates the closeness of the match of a given Mantissa Distribution to Benford's Law. And we show that this metric is deeply connected to various task-specific metrics such as accuracy. We also make a connection of MLH to overfitting and how MLH oscillates around a specific value, which may well be the optimal MLH for a Neural Network.

References

[Acebo and Sbert, 2005] E Acebo and Mateu Sbert. Benford's law for natural and synthetic images. In *Proceedings of the First Eurographics conference on Computational Aesthetics in Graphics, Visualization and Imaging*, pages 169–176, 2005.

[Benford, 1938] Frank Benford. The law of anomalous numbers. *Proceedings of the American Philosophical Society*, 78(4):551–572, 1938.

[Bonettini *et al.*, 2020] Nicolò Bonettini, Paolo Bestagini, Simone Milani, and Stefano Tubaro. On the use of benford's law to detect gan-generated images. *arXiv preprint arXiv:2004.07682*, 2020.

[Clark *et al.*, 2020] Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. Electra:

Pre-training text encoders as discriminators rather than generators. *arXiv preprint arXiv:2003.10555*, 2020.

[Contributors, 2020] MMAction2 Contributors. Openmmlab's next generation video understanding toolbox and benchmark. `https://github.com/open-mmlab/mmaction2`, 2020.

[Corral *et al.*, 2015] Álvaro Corral, Gemma Boleda, and Ramon Ferrer-i Cancho. Zipf's law for word frequencies: Word forms versus lemmas in long texts. *PLOS ONE*, 10(7):1–23, 07 2015.

[Dai *et al.*, 2020] Zihang Dai, Guokun Lai, Yiming Yang, and Quoc V Le. Funnel-transformer: Filtering out sequential redundancy for efficient language processing. *arXiv preprint arXiv:2006.03236*, 2020.

[Devlin *et al.*, 2018] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[Falcon, 2019] WA Falcon. Pytorch lightning. *GitHub. Note: https://github.com/PyTorchLightning/pytorch-lightning*, 3, 2019.

[Feichtenhofer *et al.*, 2019] Christoph Feichtenhofer, Haoqi Fan, Jitendra Malik, and Kaiming He. Slowfast networks for video recognition. In *Proceedings of the IEEE international conference on computer vision*, pages 6202–6211, 2019.

[Girshick, 2015] Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 1440–1448, 2015.

[Goodfellow *et al.*, 2013] Ian J Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, and Yoshua Bengio. An empirical investigation of catastrophic forgetting in gradient-based neural networks. *arXiv preprint arXiv:1312.6211*, 2013.

[He *et al.*, 2016] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[He *et al.*, 2017] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.

[Hill, 1995a] Theodore P. Hill. Base-invariance implies benford's law. *Proceedings of the American Mathematical Society*, 123(3):887–895, 1995.

[Hill, 1995b] Theodore P Hill. Base-invariance implies benford's law. *Proceedings of the American Mathematical Society*, 123(3):887–895, 1995.

[Hochreiter and Schmidhuber, 1997a] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[Hochreiter and Schmidhuber, 1997b] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[Huang *et al.*, 2017] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.

[Iandola *et al.*, 2016] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and¡ 0.5 mb model size. *arXiv preprint arXiv:1602.07360*, 2016.

[Iandola *et al.*, 2020] Forrest N Iandola, Albert E Shaw, Ravi Krishna, and Kurt W Keutzer. Squeezebert: What can computer vision teach nlp about efficient neural networks? *arXiv preprint arXiv:2006.11316*, 2020.

[Jolion, 2001] Jean-Michel Jolion. Images and benford's law. *Journal of Mathematical Imaging and Vision*, 14(1):73–81, 2001.

[Kingma and Ba, 2014] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[Krizhevsky and others, 2009] Alex Krizhevsky et al. Learning multiple layers of features from tiny images. 2009.

[Krizhevsky *et al.*, 2017] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, 2017.

[Lan *et al.*, 2019] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*, 2019.

[LeCun and Cortes, 2010] Yann LeCun and Corinna Cortes. MNIST handwritten digit database. 2010.

[LeCun *et al.*, 1998] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[Lin *et al.*, 2017] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017.

[Lin *et al.*, 2019] Ji Lin, Chuang Gan, and Song Han. Tsm: Temporal shift module for efficient video understanding. In *Proceedings of the IEEE International Conference on Computer Vision*, 2019.

[Ma *et al.*, 2018] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In *Proceedings of the European conference on computer vision (ECCV)*, pages 116–131, 2018.

[Mahsereci *et al.*, 2017] Maren Mahsereci, Lukas Balles, Christoph Lassner, and Philipp Hennig. Early stopping without a validation set. *CoRR*, abs/1703.09580, 2017.

[Paszke *et al.*, 2019] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.

[Pearson, 1895] Karl Pearson. Vii. note on regression and inheritance in the case of two parents. *proceedings of the royal society of London*, 58(347-352):240–242, 1895.

[Ren *et al.*, 2016] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE transactions on pattern analysis and machine intelligence*, 39(6):1137–1149, 2016.

[Sambridge *et al.*, 2010] Malcolm Sambridge, Hrvoje Tkalčić, and A Jackson. Benford's law in the natural sciences. *Geophysical research letters*, 37(22), 2010.

[Sandler *et al.*, 2018] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.

[Sanh *et al.*, 2019] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[Shao and Ma, 2010] Lijing Shao and Bo-Qiang Ma. The significant digit law in statistical physics. *Physica A: Statistical Mechanics and its Applications*, 389:3109–3116, 05 2010.

[Simonyan and Zisserman, 2014] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[Song *et al.*, 2020] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. Mpnet: Masked and permuted pre-training for language understanding. *arXiv preprint arXiv:2004.09297*, 2020.

[Sun *et al.*, 2020] Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. Mobilebert: a compact task-agnostic bert for resource-limited devices. *arXiv preprint arXiv:2004.02984*, 2020.

[Szegedy *et al.*, 2015] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.

[Szegedy *et al.*, 2016] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.

[Tan *et al.*, 2019] Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V Le. Mnasnet: Platform-aware neural architecture search for mobile. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2820–2828, 2019.

[Tran *et al.*, 2018] Du Tran, Heng Wang, Lorenzo Torresani, Jamie Ray, Yann LeCun, and Manohar Paluri. A closer look at spatiotemporal convolutions for action recognition, 2018.

[Wang *et al.*, 2016] Limin Wang, Yuanjun Xiong, Zhe Wang, Yu Qiao, Dahua Lin, Xiaoou Tang, and Luc Van Gool. Temporal segment networks: Towards good practices for deep action recognition. In *European conference on computer vision*, pages 20–36. Springer, 2016.

[Wang *et al.*, 2018] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.

[Wolf *et al.*, 2020] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics.

[Wu *et al.*, 2019] Yuxin Wu, Alexander Kirillov, Francisco Massa, Wan-Yen Lo, and Ross Girshick. Detectron2. `https://github.com/facebookresearch/detectron2`, 2019.

[Xiao *et al.*, 2017] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

[Xie *et al.*, 2017] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1492–1500, 2017.

[Zagoruyko and Komodakis, 2016] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. *arXiv preprint arXiv:1605.07146*, 2016.

7 Appendix

7.1 Preprocessing of Data

For all experiments, we only divide by the maximum magnitude across features rather than normalizing them. This was to preserve the *Scale Invariance* Property of Benford's Law.

7.2 Model Architectures

For all experiments in the paper, we use the following architectures and their respective datasets.

Dataset	Architecture
MNIST	LeNet
FashionMNIST	LeNet
CIFAR10	DenseNet-121
SequentialMNIST	LSTM
Synthetic-Uniform	MLP
Synthetic-Boolean	MLP

Table 2

The details of the models that have been harnessed for the experiments are given below:

- LeNet [LeCun *et al.*, 1998] architecture is a simple network with two(2) blocks of Strided Convolution-LeakyReLU Network followed by a fully-connected classification layer.
- DenseNet-121 [Huang *et al.*, 2017] implementation is borrowed from this GitHub repository[2].
- LSTM is a simple 2-layered RNN with Long-Short Term Memory [Hochreiter and Schmidhuber, 1997b] followed by a fully-connected layer for classification.
- MLP is a 2 fully-connected network with Classification (Softmax) and Regression (Linear) output layers for Synthetic-Boolean and Synthetic Uniform Datasets.

7.3 Hyperparameters

Unless stated otherwise, all of the experiments use PyTorch's [Paszke *et al.*, 2019] implementation of Adam [Kingma and Ba, 2014], and PyTorch Lightning's [Falcon, 2019] default values except the following wherever applicable.

Parameter	Value
Learning Rate	3e-3
Early Stopping Patience	15
Validation Frequency	0.33
Batch Size	64

Table 3

7.4 Data Splits

For the datasets other than synthetically generated, we provide the Train/Validation/Test sets. All of them are randomly split unless PyTorch only has Train/Test splits.

[2] https://github.com/kuangliu/pytorch-cifar/blob/master/models/densenet.py

Dataset	Train	Validation	Test
MNIST	45000	5000	10000
FashionMNIST	45000	5000	10000
CIFAR10	45000	5000	10000
SequentialMNIST	45000	5000	10000
Synthetic-Boolean	6000	2000	2000
Synthetic-Uniform	6000	2000	2000

Table 4

7.5 Synthetic Data Generation

We use synthetic data set for training a regression and a classification model. The synthetic data is generated using a single layered neural network that is randomly initialized. We describe the process of the generation process below. Both the data sets have an input vector length of 64 and these input vectors are also drawn randomly from Uniform $[0, 1]$ and Bernoulli for Synthetic-Uniform and Synthetic-Boolean respectively. We generate 10000 pairs of input-output. These are split in the ratio 60:20:20 for Train, Validation and Test sets respectively.

7.6 Regression data set

$x_i \in \mathbf{R}^{64}$ is an input vector sampled from $Uniform[0, 1]$. The target label $y_i \in \mathbf{R}$ is obtained by feeding x_i as input to a randomly initialized single layered linear kernel with one output unit, f_θ.

$$y_i = f_\theta(x_i), x_i \in Uniform[0, 1] \tag{5}$$

7.7 Classification data set

$x_i \in \mathbf{R}^{64}$ is an input vector sampled from $Beroulli[0, 1]$. The target label $y_i \in \mathbf{R}$ is obtained by feeding x_i as input to a randomly initialized single layered linear kernel with two output units and argmax-ed over the output vector.

$$y_i = argmax[f_\theta(x_i)], x_i \in Bernoulli[0, 1] \tag{6}$$

8 MLH PyTorch Code

We provide an efficient PyTorch implementation to compute the MLH for a given flattened real-valued tensor.

```
benford = np.array([30.1, 17.6, 12.5, 9.7,
                    7.9, 6.7, 5.8, 5.1, 4.6]
            ) / 100

def mlh(bin_percent):
    return scipy.stats.pearsonr(
                benford,
                bin_percent[1:]
    )[0]

def bincount(tensor):
    counts = torch.zeros(10)
    for i in range(10):
        counts[i] = torch.count_nonzero(
            tensor == i
        )
    return counts

```

```
@torch.no_grad()
def bin_percent(tensor):
    tensor = tensor.abs() * 1e10
    long_tensor = torch.log10(tensor).long()
    tensor = tensor // 10 ** long_tensor
    tensor = bincount(tensor.long())
    return tensor / tensor.sum()
```

In the above code, the vector $benford$ is the distribution defined by Benford's Law, $bin_percent$ is a function that takes a tensor as input and returns the distribution of digits in it. $bintcount$ is a function that takes a flattened tensor of integers $\in [0, 9]$ and returns a vector of frequencies. mlh is the function to compute the proposed MLH score.

8.1 Generalization Experiments

To establish that neural networks indeed follow Benford's Law, we choose several pre-trained models and their variants. Please find the exhaustive list of models used below:

- Bert [Devlin *et al.*, 2018]
- DistilBERT [Sanh *et al.*, 2019]
- Funnel-Transformer [Dai *et al.*, 2020]
- Mobilebert [Sun *et al.*, 2020]
- SqueezeBERT [Iandola *et al.*, 2020]
- Electra [Clark *et al.*, 2020]
- Mpnet [Song *et al.*, 2020]
- Albert [Lan *et al.*, 2019]
- Temporal Segment Networks [Wang *et al.*, 2016]
- TSM [Lin *et al.*, 2019]
- Fast r-cnn [Girshick, 2015]
- Faster r-cnn [Ren *et al.*, 2016]
- mask r-cnn [He *et al.*, 2017]
- Focal Loss for dense object detection (RetinaNet) [Lin *et al.*, 2017]
- R2Plus1D [Tran *et al.*, 2018]
- slowonly [Feichtenhofer *et al.*, 2019]
- slowfast [Feichtenhofer *et al.*, 2019]
- MobileNet-v2 [Sandler *et al.*, 2018]
- AlexNet [Krizhevsky *et al.*, 2017]
- SqueezeNet [Iandola *et al.*, 2016]
- Mnasnet [Tan *et al.*, 2019]
- GoogLeNet [Szegedy *et al.*, 2015]
- ResNext [Xie *et al.*, 2017]
- Wide-ResNet50 [Zagoruyko and Komodakis, 2016]
- Shufflenet v2 [Ma *et al.*, 2018]
- ResNet-18 [He *et al.*, 2016]
- VGG-16 [Simonyan and Zisserman, 2014]
- Inception v3 [Szegedy *et al.*, 2016]
- DenseNet-161 [Huang *et al.*, 2017]

Task	Model Name	Metric	Value	MLH
Image Classification	ResNet18	ACC	0.69	0.99
	AlexNet		0.56	0.96
	SqueezeNet_1.0		0.58	0.97
	VGG16		0.71	0.99
	DenseNet161		0.77	0.99
	InceptionV3		0.77	0.99
	GoogleNet		0.69	0.98
	ShuffleNetV2		0.69	0.98
	MobileNetV2		0.71	0.96
	ResNeXt50		0.77	0.98
	WideResNet50		0.78	0.98
	MNASNet_1.0		0.73	0.96
Masked LM	ALBERT	GLUE	0.89	0.98
	BERT-Base		0.8	0.94
	DistillBert		0.77	0.95
	Electra-small		0.85	0.98
	MPNET		0.87	0.99
	FunnelTransformer		0.86	0.99
	MobileBert		0.78	0.97
	SqueezeBert		0.78	0.99
Text Generation	GPT2	PPL	0.35	0.97
	DARTS		0.56	0.98
	TransformerXL		0.54	0.99
	FRAGES		0.46	0.99
	DRILL		0.49	0.99
	DirectOC		0.47	0.98
Object Detection	fast_rcnn_R_50_FPN_1x	box_AP	0.37	0.98
	faster_rcnn_R_101_C4_3x		0.41	0.99
	faster_rcnn_R_101_DC5_3x		0.4	0.99
	faster_rcnn_R_101_FPN_3x		0.42	0.98
	faster_rcnn_R_50_C4_1x		0.35	0.95
	faster_rcnn_R_50_C4_3x		0.38	0.99
	faster_rcnn_R_50_DC5_1x		0.37	0.99
	faster_rcnn_R_50_DC5_3x		0.39	0.99
	faster_rcnn_R_50_FPN_1x		0.37	0.98
	faster_rcnn_R_50_FPN_3x		0.4	0.98
	faster_rcnn_X_101_32x8d_FPN_3x		0.43	0.98
	retinanet_R_101_FPN_3x		0.4	0.99
	retinanet_R_50_FPN_1x		0.37	0.98
	retinanet_R_50_FPN_3x		0.38	0.99
Video Classification	tsn_r50_1x1x3_100e_kinetics400_rgb	ACC	0.7	0.98
	tsn_r50_dense_1x1x5_50e_kinetics400_rgb		0.7	0.98
	tsn_r50_320p_1x1x3_100e_kinetics400_rgb		0.7	0.98
	tsn_r50_320p_1x1x3_110e_kinetics400_flow		0.55	0.98
	tsn_r50_1x1x8_100e_kinetics400_rgb		0.71	0.98
	tsn_r50_320p_1x1x8_100e_kinetics400_rgb		0.72	0.98
	tsn_r50_320p_1x1x8_110e_kinetics400_flow		0.57	0.98
	tsn_r50_video_320p_1x1x3_100e_kinetics400_rgb		0.71	0.98
	tsn_r50_dense_1x1x8_100e_kinetics400_rgb		0.7	0.99
	tsn_r50_video_1x1x8_100e_kinetics400_rgb		0.71	0.98
	tsn_r50_video_dense_1x1x8_100e_kinetics400_rgb		0.7	0.98
	tsm_r50_1x1x8_50e_kinetics400_rgb		0.7	0.99
	tsm_r50_video_1x1x8_50e_kinetics400_rgb		0.7	0.99
	tsm_r50_dense_1x1x8_100e_kinetics400_rgb		0.72	0.98
	tsm_r50_1x1x16_50e_kinetics400_rgb		0.72	0.99
	tsm_nl_embedded_gaussian_r50_1x1x8_50e_kinetics400_rgb		0.72	0.99
	tsm_nl_gaussian_r50_1x1x8_50e_kinetics400_rgb		0.7	0.99

Task	Model Name	Metric	Value	MLH
Video Classification	tsm_nl_dot_product_r50_1x1x8_50e_kinetics400_rgb	ACC	0.71	0.99
	i3d_r50_32x2x1_100e_kinetics400_rgb		0.72	0.99
	i3d_r50_video_32x2x1_100e_kinetics400_rgb		0.72	0.99
	i3d_r50_dense_32x2x1_100e_kinetics400_rgb		0.72	0.99
	i3d_r50_lazy_32x2x1_100e_kinetics400_rgb		0.72	0.99
	i3d_nl_embedded_gaussian_r50_32x2x1_100e_kinetics400_rgb		0.74	0.99
	i3d_nl_gaussian_r50_32x2x1_100e_kinetics400_rgb		0.73	0.99
	i3d_nl_dot_product_r50_32x2x1_100e_kinetics400_rgb		0.73	0.99
	r2plus1d_r34_8x8x1_180e_kinetics400_rgb		0.67	0.99
	r2plus1d_r34_video_8x8x1_180e_kinetics400_rgb		0.67	0.99
	r2plus1d_r34_32x2x1_180e_kinetics400_rgb		0.74	0.99
	slowonly_r50_4x16x1_256e_kinetics400_rgb		0.72	0.98
	slowonly_r50_video_4x16x1_256e_kinetics400_rgb		0.72	0.99
	slowonly_r50_8x8x1_256e_kinetics400_rgb		0.74	0.98
	slowonly_r50_4x16x1_256e_kinetics400_rgb		0.73	0.99
	slowonly_r50_8x8x1_256e_kinetics400_rgb		0.74	0.98
	slowonly_imagenet_pretrained_r50_4x16x1_150e_kinetics400_rgb		0.73	0.99
	slowonly_imagenet_pretrained_r50_8x8x1_150e_kinetics400_rgb		0.75	0.99
	slowonly_r50_4x16x1_256e_kinetics400_flow		0.61	0.98
	slowonly_r50_8x8x1_196e_kinetics400_flow		0.65	0.98
	slowfast_r50_4x16x1_256e_kinetics400_rgb		0.74	0.99
	slowfast_r50_video_4x16x1_256e_kinetics400_rgb		0.74	0.98
	slowfast_r50_4x16x1_256e_kinetics400_rgb		0.75	0.98
	tsn_r50_1x1x3_100e_kinetics400_rgb		0.7	0.98
	tsn_r50_dense_1x1x5_50e_kinetics400_rgb		0.7	0.98
	tsn_r50_320p_1x1x3_100e_kinetics400_rgb		0.7	0.98
	tsn_r50_320p_1x1x3_110e_kinetics400_flow		0.55	0.98
	tsn_r50_1x1x8_100e_kinetics400_rgb		0.71	0.98
	tsn_r50_320p_1x1x8_100e_kinetics400_rgb		0.72	0.98
Instance Segmentation	mask_rcnn_R_101_C4_3x	boxAP	0.42	0.99
	mask_rcnn_R_101_DC5_3x		0.41	0.99
	mask_rcnn_R_101_FPN_3x		0.42	0.98
	mask_rcnn_R_50_C4_1x		0.36	0.96
	mask_rcnn_R_50_C4_3x		0.39	0.99
	mask_rcnn_R_50_DC5_1x		0.38	0.99
	mask_rcnn_R_50_DC5_3x		0.4	0.99
	mask_rcnn_R_50_FPN_1x		0.38	0.98
	mask_rcnn_R_50_FPN_3x		0.41	0.98
	mask_rcnn_X_101_32x8d_FPN_3x		0.44	0.98
Instance Segmentation	mask_rcnn_R_101_C4_3x	maskAP	0.36	0.99
	mask_rcnn_R_101_DC5_3x		0.37	0.99
	mask_rcnn_R_101_FPN_3x		0.38	0.98
	mask_rcnn_R_50_C4_1x		0.32	0.96
	mask_rcnn_R_50_C4_3x		0.34	0.99
	mask_rcnn_R_50_DC5_1x		0.34	0.99
	mask_rcnn_R_50_DC5_3x		0.35	0.99
	mask_rcnn_R_50_FPN_1x		0.35	0.98
	mask_rcnn_R_50_FPN_3x		0.37	0.98
	mask_rcnn_X_101_32x8d_FPN_3x		0.39	0.98

Table 5: List of variants of pre-trained models used for generalization experiment.